



Natural Language Processing: Word Embeddings

HSE Faculty of Computer Science
Machine Learning and Data-Intensive Systems

Murat Khazhgeriev



Table of Content

- Organizational matters
- Preprocessing pipeline
- But what is a Word Embedding?
- Statistics-based approaches
- Deep Learning approaches
- Useful facts



Table of Content

- **Organizational matters**
 - Homework & grade policy
 - Resources
- Preprocessing pipeline
- But what is a Word Embedding?
- Statistics-based approaches
- Deep Learning approaches
- Useful facts



Table of Content

- Organizational matters
 - **Homework & grade policy**
 - Resources
- Preprocessing pipeline
- But what is a Word Embedding?
- Statistics-based approaches
- Deep Learning approaches
- Useful facts



Grade policy

70% (homework) + 30% (exam)

Homework

- (20%) Week 2. Training embeddings using the fasttext library, implementation of a real search engine for embedding-response upon request in a vector database.
- (30%) Week 4. Fine-tuning BERT on your own data, training GPT from scratch
- (20%) Week 5: Fine tuning LLM using PEFT.



Table of Content

- Organizational matters
 - Homework & grade policy
 - **Resources**
- Preprocessing pipeline
- But what is a Word Embedding?
- Statistics-based approaches
- Deep Learning approaches
- Useful facts



Course materials

- [Syllabus \(Notion\)](#)
- [Github](#)
- [HSE Wiki](#)

Useful sources

- [NLP Course For You](#)
- [YSDA NLP Course](#)
- [CS224n](#)



Table of Content

- Organizational matters
- **Preprocessing pipeline**
 - Tokenization
 - Lowering, Punctuation, Stop Words, Filtration
 - Normalization
- But what is a Word Embedding?
- Statistics-based approaches
- Deep Learning approaches
- Useful facts



Table of Content

- Organizational matters
- Preprocessing pipeline
 - **Tokenization**
 - Lowering, Punctuation, Stop Words, Filtration
 - Normalization
- But what is a Word Embedding?
- Statistics-based approaches
- Deep Learning approaches
- Useful facts



Word-Level Tokenization

"ChatGPT is a powerful AI tool." $\longrightarrow$ ["ChatGPT", "is", "a", "powerful", "AI", "tool", "."]

Character-Level Tokenization

"ChatGPT is a powerful AI tool." $\longrightarrow$ ["C", "h", "a", "t", "G", "P", "T", " ", "i", "s", " ",
",", "a", " ", "p", "o", "w", "e", "r", "f", "u", "l",
" ", "A", "I", " ", "t", "o", "o", "l", "."]

$a, -, c, -ab$

Byte-Pair Encoding (BPE) Tokenization

"ChatGPT is a powerful AI tool." $\longrightarrow$ ["Chat", "GP", "T", "is", "a", "power", "ful",
"AI", "tool", "."]

$abacaba$
 $\underbrace{\quad\quad\quad}$

0. $ab = 2$
 $ba = 2$
 $ac = 1$
...

1. $aba = 2$ 2. $abac$
 $ac = 1$
...

$\left. \begin{array}{l} a + b \rightarrow ab \\ ab + a \rightarrow aba \end{array} \right\}$

$acab$
 $\underbrace{\quad\quad\quad}$

a, c, ab

Table of Content

- Organizational matters
- Preprocessing pipeline
 - Tokenization
 - **Lowering, Punctuation, Stop Words, Filtration**
 - Normalization
- But what is a Word Embedding?
- Statistics-based approaches
- Deep Learning approaches
- Useful facts



Preprocessing pipeline

Lowering, Punctuation, Stop
Words, Filtration

"The quick brown fox jumps over the lazy dog!"



Lowering: "the quick brown fox jumps over the lazy dog!"



Punctuation removal: "the quick brown fox jumps over the lazy dog"



Stop Words Removal: "quick brown fox jumps lazy dog"



Table of Content

- Organizational matters
- Preprocessing pipeline
 - Tokenization
 - Lowering, Punctuation, Stop Words, Filtration
 - **Normalization**
- But what is a Word Embedding?
- Statistics-based approaches
- Deep Learning approaches
- Useful facts



“Динозавры играют в большой парк около школы.”

• Stemming

«Динозавр игра в больш парк около школ.»

(аигл.)

• Lemmatization

«Динозавр играть в большой парк около школа.»

(рус.)



Table of Content

- Organizational matters
- Preprocessing pipeline
- **But what is a Word Embedding?**
 - Intuition behind
 - One-Hot Vectors
- Statistics-based approaches
- Deep Learning approaches
- Useful facts

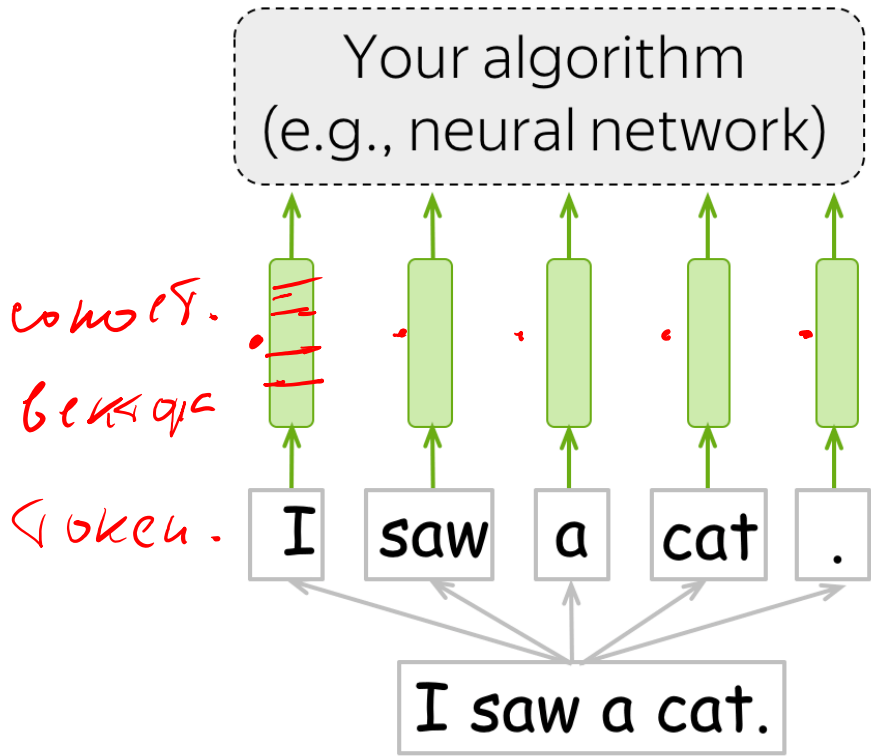


Table of Content

- Organizational matters
- Preprocessing pipeline
- But what is a Word Embedding?
 - **Intuition behind**
 - One-Hot Vectors
- Statistics-based approaches
- Deep Learning approaches
- Useful facts

Tokenize an input text for further processing

	↓ муж	↓ жен	↓ сум
→ кбарт	5	3	30
→ кбарт-1	12	2	20



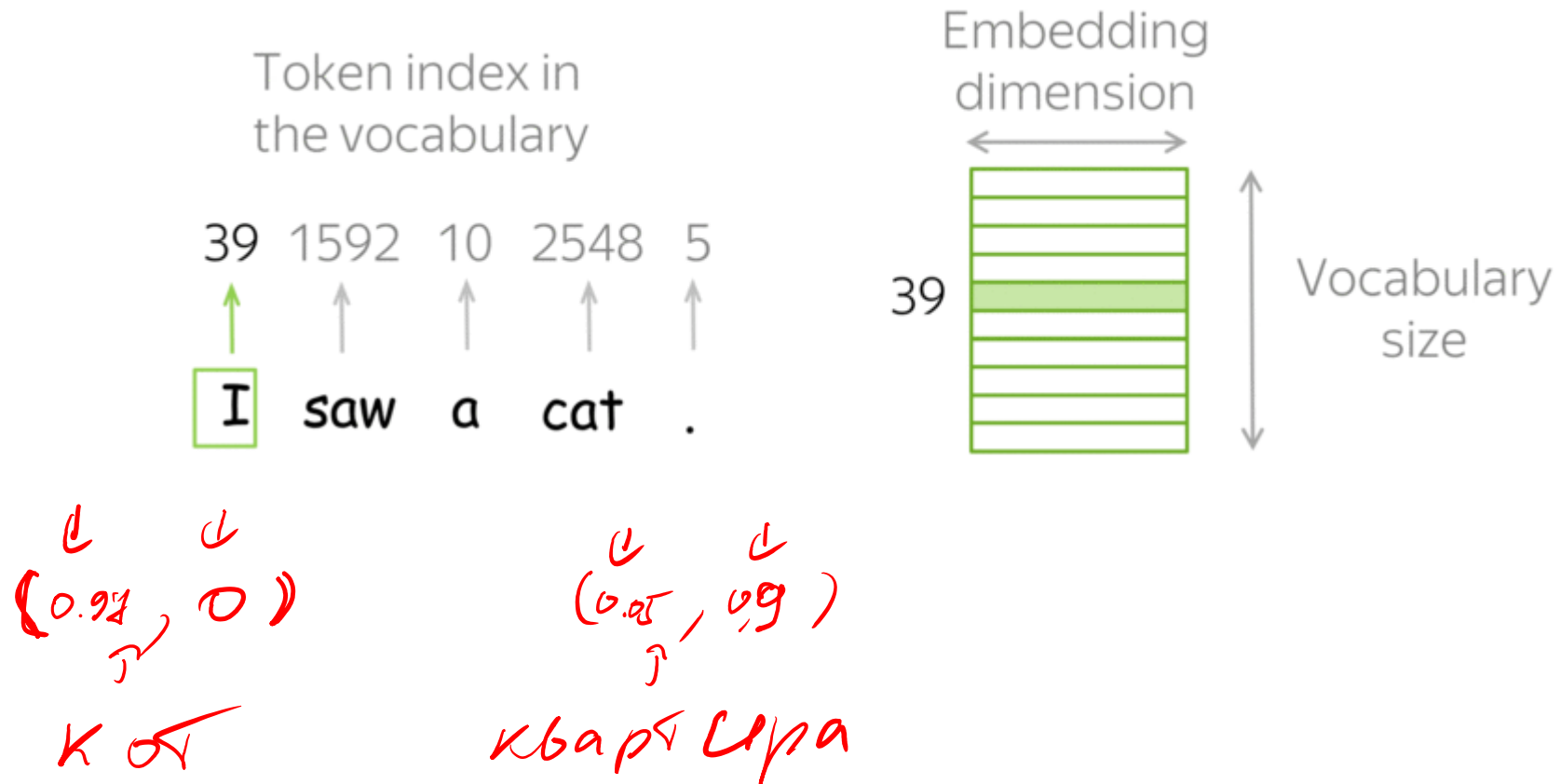
Any algorithm for solving a task

Word representation - vector
(input for your model/algorithm)

Sequence of tokens

Text (your input)

Match each token to a vector



A word's meaning is defined by its context

Now look how this word is used in different contexts:

A bottle of **tezgüino** is on the table.

Everyone likes **tezgüino**.

Tezgüino makes you drunk.

We make **tezgüino** out of corn.

Can you understand what **tezgüino** means ?

A word's meaning is defined by its context

- (1) A bottle of _____ is on the table.
- (2) Everyone likes _____ .
- (3) _____ makes you drunk.
- (4) We make _____ out of corn.

What other words fit into these contexts ?

	(1)	(2)	(3)	(4)	...	← contexts
→ tezgüino	1	1	1	1		
loud	0	0	0	0		
motor oil	1	0	0	1		
tortillas	0	1	0	1		
→ wine	1	1	1	0		

← rows show contextual properties: 1 if a word can appear in the context, 0 if not



Reserve a token for special cases e.g. unknown words

I saw a UNK .
↑ ↑ ↑ ↑ ↑
I saw a &%! .

not in the
vocabulary

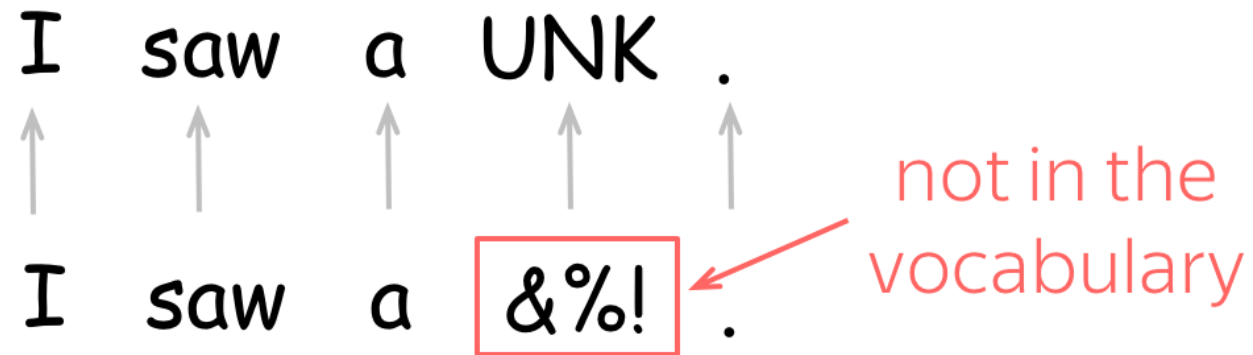
The diagram shows two rows of text. The top row is 'I saw a UNK .' and the bottom row is 'I saw a &%! .'. Vertical arrows point from each word in the bottom row to the corresponding word in the top row. The word '&%!' in the bottom row is enclosed in a red rectangular box. A red arrow points from the text 'not in the vocabulary' to this box.

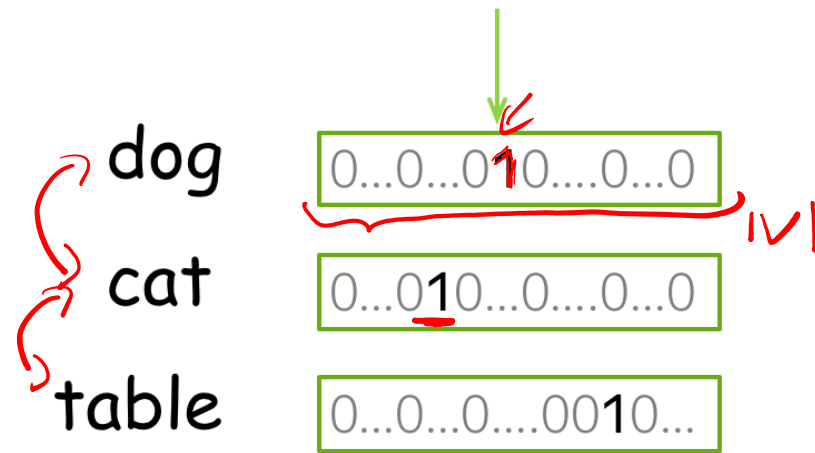


Table of Content

- Organizational matters
- Preprocessing pipeline
- But what is a Word Embedding?
 - Intuition behind
 - **One-Hot Vectors**
- Statistics-based approaches
- Deep Learning approaches
- Useful facts

The easiest way to go is One-Hot Encoding

One is 1, the rest are 0



Embedding dimension =
vocabulary size



Table of Content

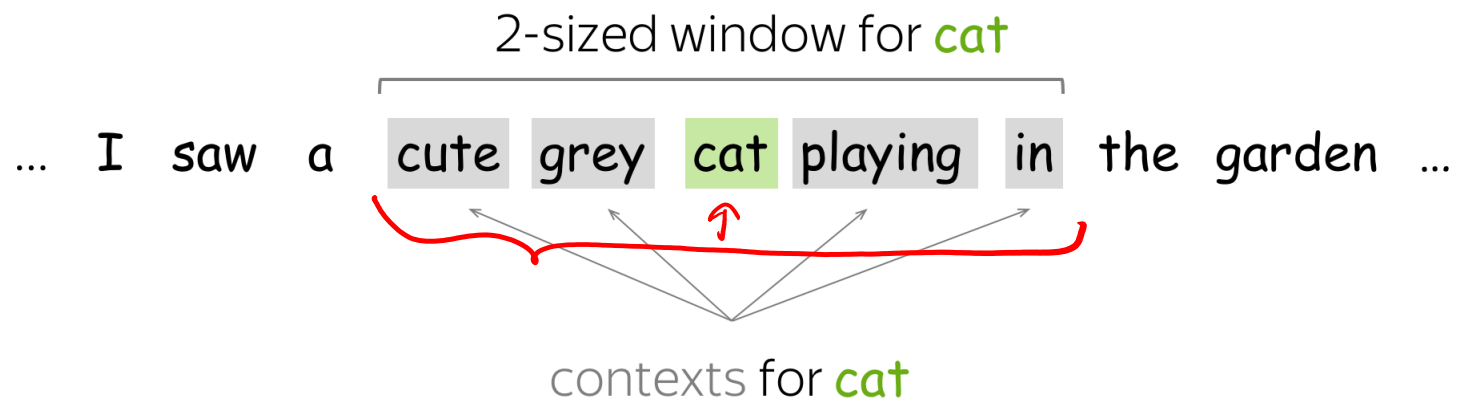
- Organizational matters
- Preprocessing pipeline
- But what is a Word Embedding?
- **Count-based (pre-neural) approaches**
 - Co-occurrence count
 - Bag-of-Words (BOW)
 - PPMI
 - TF-IDF
 - Latent Semantic Analysis
- Word2vec
- Useful facts



Table of Content

- Organizational matters
- Preprocessing pipeline
- But what is a Word Embedding?
- Count-based (pre-neural) approaches
 - **Co-occurrence count**
 - Bag-of-Words (BOW)
 - PPMI
 - TF-IDF
 - Latent Semantic Analysis
- Word2vec
- Useful facts

Define context via a window in a text



	t_0	t_1	t_2	t_3	...	cat	c
t_0							
t_1							
t_2							
t_3							
...							
cat							
...							

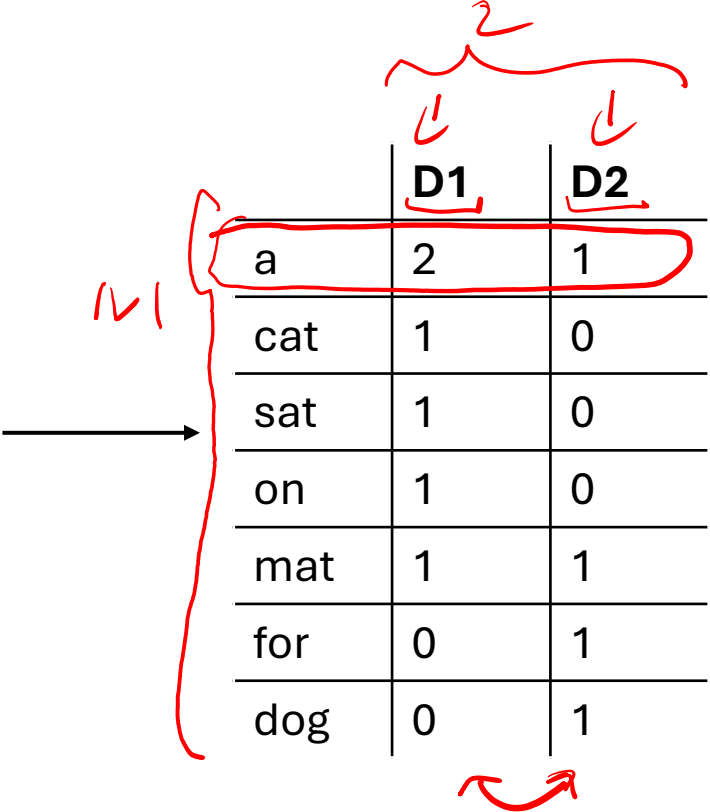


Table of Content

- Organizational matters
- Preprocessing pipeline
- But what is a Word Embedding?
- Count-based (pre-neural) approaches
 - Co-occurrence count
 - **Bag-of-Words (BOW)**
 - PPMI
 - TF-IDF
 - BM25
 - Latent Semantic Analysis
- Word2vec
- Useful facts

We can also treat the whole document as a context

D1: a cat sat on a mat
D2: a mat for a dog



	D1	D2
a	2	1
cat	1	0
sat	1	0
on	1	0
mat	1	1
for	0	1
dog	0	1

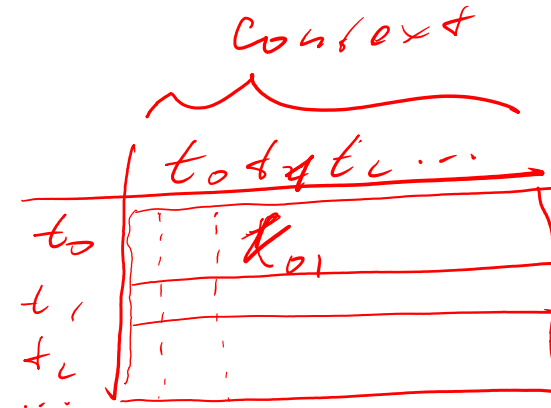
Table of Content

- Organizational matters
- Preprocessing pipeline
- But what is a Word Embedding?
- Count-based (pre-neural) approaches
 - Co-occurrence count
 - Bag-of-Words (BOW)
 - **PPMI**
 - TF-IDF
 - BM25
 - Latent Semantic Analysis
- Word2vec
- Useful facts

Positive Pointwise Mutual Information

Context:

- surrounding words
in a L-sized window



$$\sum_i \sum_j k_{ij} = N$$

Matrix element:

- $\text{PPMI}(\mathbf{w}, c) = \max(0, \text{PMI}(\mathbf{w}, c))$,

where

$$\text{PMI}(\mathbf{w}, c) = \log \frac{P(\mathbf{w}, c)}{P(\mathbf{w})P(c)} = \log \frac{N(\mathbf{w}, c)|(\mathbf{w}, c)|}{N(\mathbf{w})N(c)}$$

$$P(w) = \frac{N(w)}{N}$$

$$P(c) = \frac{N(c)}{N}$$

$$P(w, c) = \frac{k_{wc}}{N}$$

$$= \frac{N(w, c)}{N}$$

$$= \log \frac{N(w, c) \cdot N}{N(w) N(c)}$$



Table of Content

- Organizational matters
- Preprocessing pipeline
- But what is a Word Embedding?
- Count-based (pre-neural) approaches
 - Co-occurrence count
 - Bag-of-Words (BOW)
 - PPMI
 - **TF-IDF**
 - BM25
 - Latent Semantic Analysis
- Word2vec
- Useful facts

We can also account for a term being widespread

Context:

- document d (from a collection D)

Matrix element:

- $\text{tf-idf}(\underline{w}, \underline{d}, \underline{D}) = \text{tf}(\underline{w}, \underline{d}) \cdot \text{idf}(\underline{w}, \underline{D})$

$N(\underline{w}, \underline{d})$

term frequency

$$\log \frac{|D|}{|\{d \in D: \underline{w} \in d\}|}$$

inverse document frequency



Table of Content

- Organizational matters
- Preprocessing pipeline
- But what is a Word Embedding?
- Count-based (pre-neural) approaches
 - Co-occurrence count
 - Bag-of-Words (BOW)
 - PPMI
 - TF-IDF
 - **BM25**
 - Latent Semantic Analysis
- Word2vec
- Useful facts

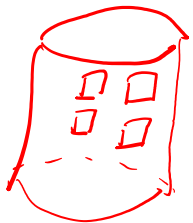
BM25 (best matching 25)

$$\text{score}(\underline{D}, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{\overset{\text{TF}}{\underset{\text{TF}}{f(q_i, D)}} \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

$$f(q_i, D) = f$$

$$\frac{f \cdot (k_1 + 1)}{f + k_1 \cdot b}$$

$$\text{query} = Q = \{q_0, q_1, \dots\}$$



Doc corpus

$$\text{IDF}(q_i) = \ln \left(\frac{\overset{|Corpus|}{N - n(q_i)} + 0.5}{\underline{n(q_i)} + 0.5} + 1 \right)$$

$$n(q_i) = |\{ \text{Doc corpus} \mid D \ni q_i \}|$$



Table of Content

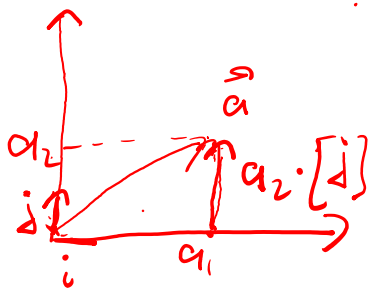
- Organizational matters
- Preprocessing pipeline
- But what is a Word Embedding?
- Count-based (pre-neural) approaches
 - Co-occurrence count
 - Bag-of-Words (BOW)
 - PPMI
 - TF-IDF
 - BM25
 - **Latent Semantic Analysis**
- Word2vec
- Useful facts

Matrix factorization is a way to get dense embeddings

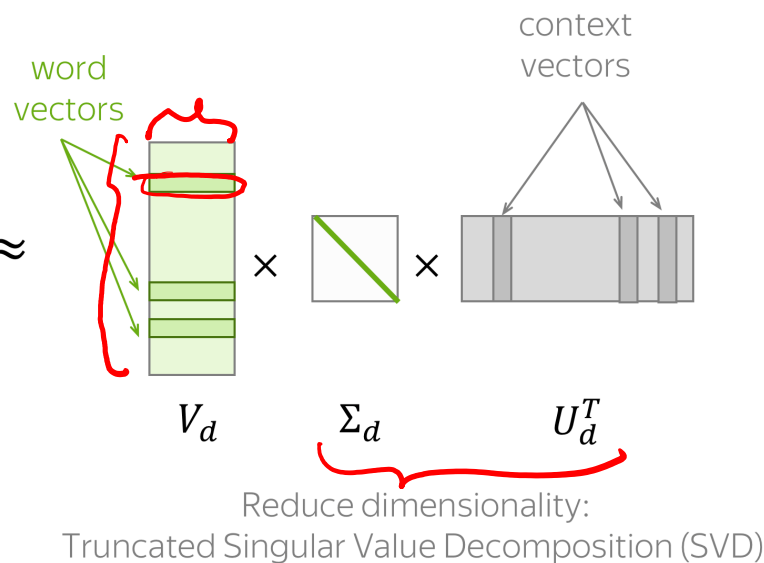
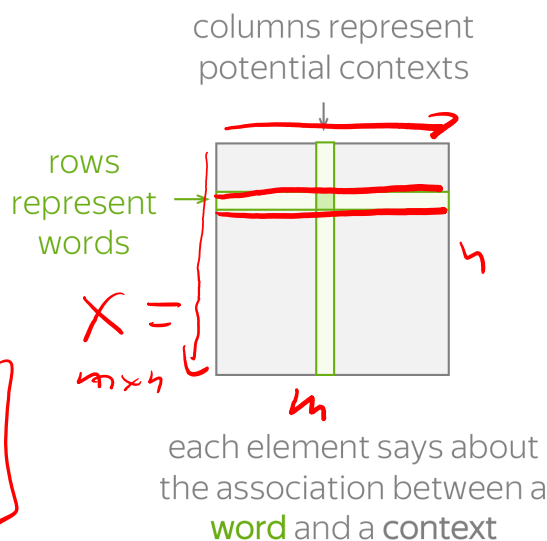
$$\begin{matrix} & t_1 & t_2 & t_3 & \dots & t_m \\ t_i & \begin{bmatrix} 0 & 0 & k_{i3} & \dots & 0 \end{bmatrix} \end{matrix}$$

$$X = V \Sigma U^T$$

$$ALS = \min_{m \times d, d \times n, m \times n} \|UV - X\|_F^2 \rightarrow \min$$



$$\vec{a} = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = a_1 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + a_2 \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$



$$X_{m \times n} = \underbrace{V}_{m \times d'} \cdot \underbrace{\Sigma}_{d' \times d'} \cdot \underbrace{U^T}_{d' \times n} \quad d' \leq d$$



Table of Content

- Organizational matters
- Preprocessing pipeline
- But what is a Word Embedding?
- Count-based (pre-neural) approaches
- **Word2vec**
 - Idea behind
 - Objective function
 - Training Procedure
 - Negative sampling
 - Skip-Gram vs. CBOW
 - GloVe
- Useful facts



Table of Content

- Organizational matters
- Preprocessing pipeline
- But what is a Word Embedding?
- Count-based (pre-neural) approaches
- Word2vec
 - **Idea behind**
 - Objective function
 - Training procedure
 - Negative sampling
 - Skip-Gram vs. CBOW
 - GloVe
- Useful facts

Slide one word at a time

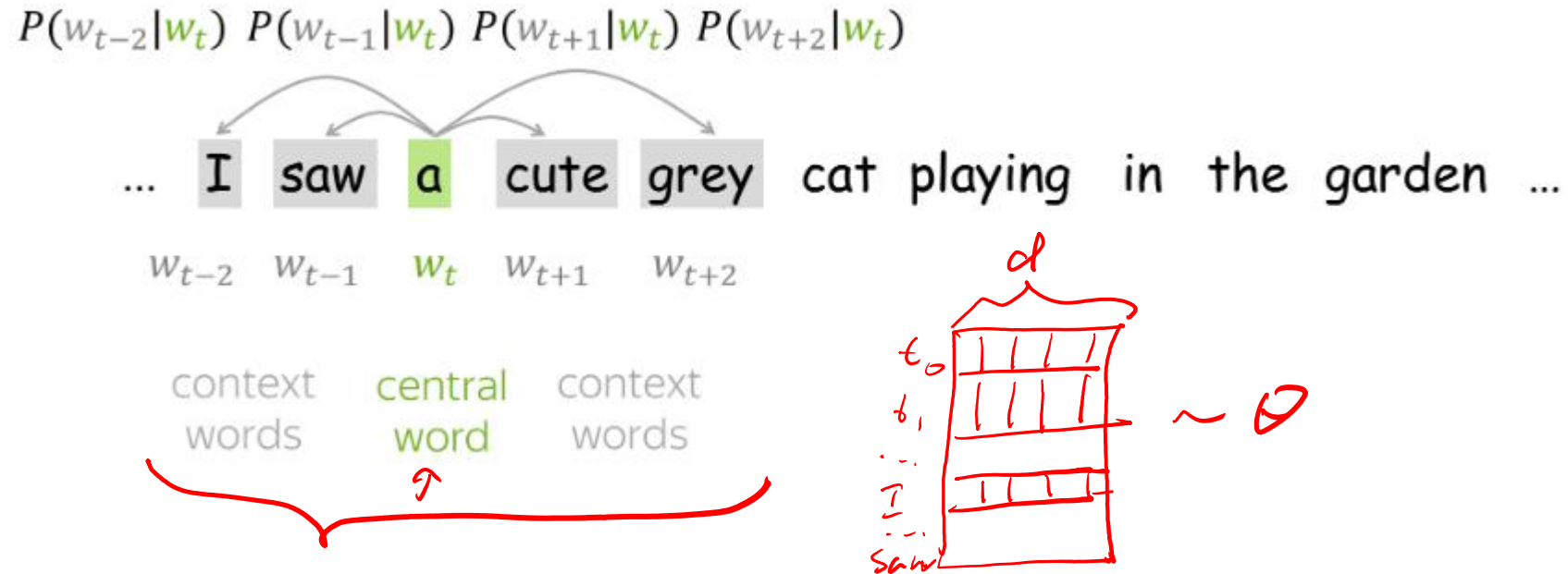


Table of Content

- Organizational matters
- Preprocessing pipeline
- But what is a Word Embedding?
- Count-based (pre-neural) approaches
- Word2vec
 - Idea behind
 - **Objective function**
 - Training procedure
 - Negative Sampling
 - Skip-Gram vs. CBOW
 - GloVe
- Useful facts

Maximize the probability of encountering a target word given context

For each position $t = 1, \dots, T$ in a text corpus, Word2Vec predicts context words within a m -sized window given the central word w_t :

$$\text{Likelihood} = L(\theta) = \prod_{t=1}^T \prod_{-m \leq j \leq m, j \neq 0} P(\underline{w_{t+j}} | \underline{w_t}, \underline{\theta}),$$

where θ are all variables to be optimized. The objective function (aka loss function or cost function) $J(\theta)$ is the average negative log-likelihood:

Loglikelihood for computational efficiency

$$\text{Loss} = J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m, \\ j \neq 0}} \log P(\underline{w_{t+j}} | \underline{w_t}, \theta)$$

agrees with our
plan above



go over text



with a sliding
window



compute probability of the
context word given the central

Loglikelihood for computational efficiency

$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

Dot product: measures similarity of o and c
Larger dot product = larger probability

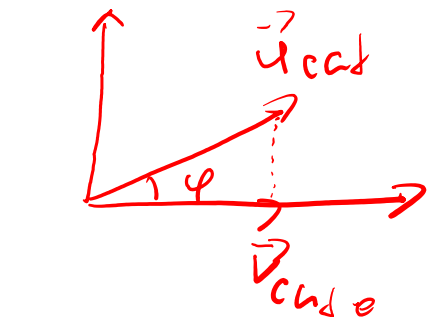
Normalize over entire vocabulary
to get probability distribution

$n \prod_{i=1}^n$
 $\vec{a} = (a_1, a_2, a_3, \dots)$

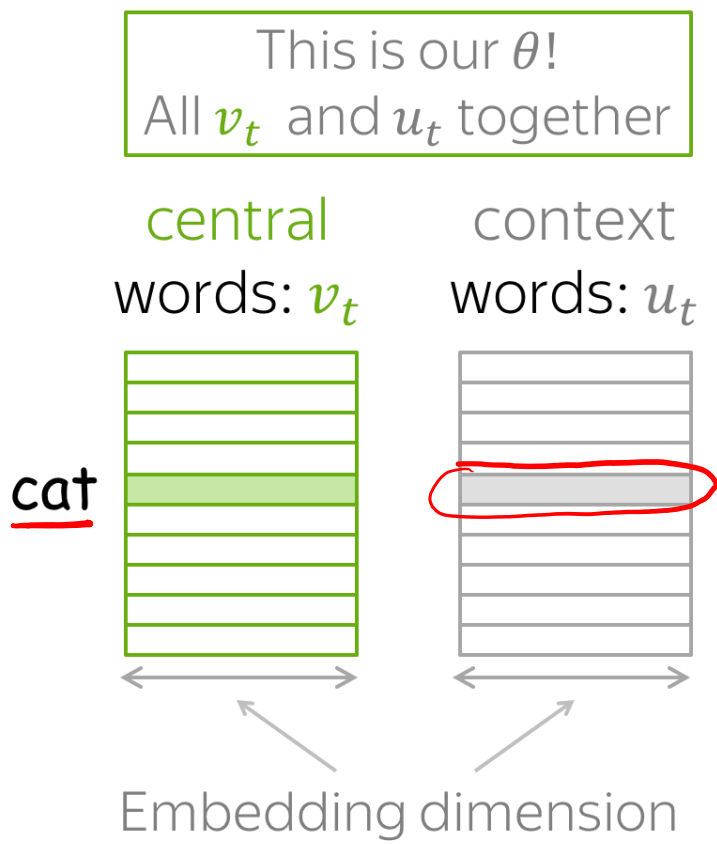
$$\text{softmax}(\vec{a}) = \frac{\exp(a_i)}{\sum \exp(a_i)}$$

Note that we have distinct embeddings for context and target cases

$\vec{v}_{cat}$ $\langle \vec{u}_{cat}, \vec{v}_{cat} \rangle$
 $\vec{u}_{cat}$



$\langle \vec{u}, \vec{v} \rangle = \cos \varphi |\vec{u}| |\vec{v}|$



$u = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}; v = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}$

$\langle \vec{u}, \vec{v} \rangle = u_1 \cdot v_1 + u_2 \cdot v_2$

$(-\vec{u}-) = (u_1, u_2) \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} =$

$= \vec{u}^T \vec{v}$

Table of Content

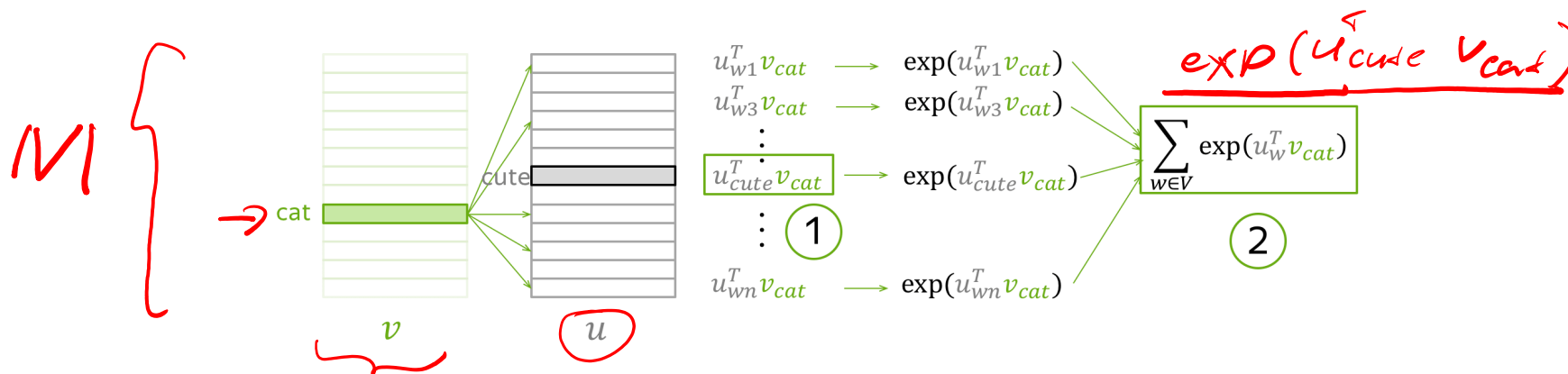
- Organizational matters
- Preprocessing pipeline
- But what is a Word Embedding?
- Count-based (pre-neural) approaches
- Word2vec
 - Idea behind
 - Objective function
 - **Training procedure**
 - Negative Sampling
 - Skip-Gram vs. CBOW
 - GloVe
- Useful facts

A schematic overview on the training procedure

1. Take dot product of v_{cat} with all u

2. exp

3. sum all



4. get loss (for this one step)

5. evaluate the gradient,
make an update

$$J_{t,j}(\theta) = \underbrace{-u_{cute}^T v_{cat}}_{1} + \log \underbrace{\sum_{w \in V} \exp(u_w^T v_{cat})}_{2}$$

$$v_{cat} := v_{cat} - \alpha \frac{\partial J_{t,j}(\theta)}{\partial v_{cat}}$$

$$u_w := u_w - \alpha \frac{\partial J_{t,j}(\theta)}{\partial u_w} \quad \forall w \in V$$



Table of Content

- Organizational matters
- Preprocessing pipeline
- But what is a Word Embedding?
- Count-based (pre-neural) approaches
- Word2vec
 - Idea behind
 - Objective function
 - Training procedure
 - **Negative Sampling**
 - Skip-Gram vs. CBOW
 - GloVe
- Useful facts

Negative sampling to speed up the computations

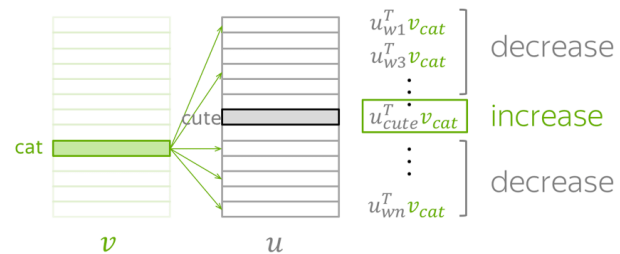
Dot product of v_{cat} :

- with u_{cute} - increase,
- with all other u - decrease



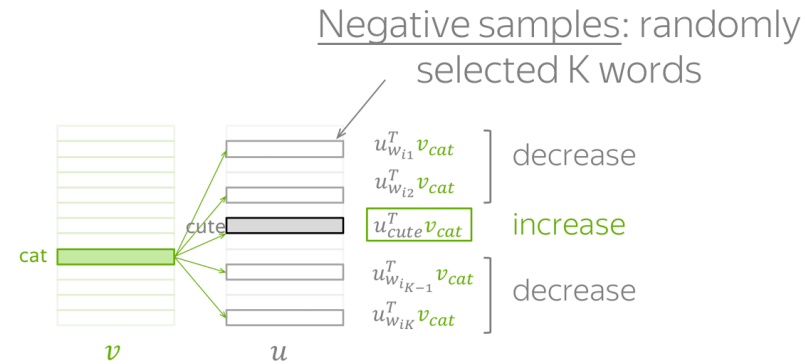
Dot product of v_{cat} :

- with u_{cute} - increase,
- with a subset of other u - decrease



Parameters to be updated:

- v_{cat}
- u_w for all w in the vocabulary $|V| + 1$ vectors



Parameters to be updated:

- v_{cat}
- u_{cute} and u_w for w in K negative examples $K + 2$ vectors

A loss function given negative sampling

$$J_{t,j}(\theta) = -\log \sigma(\underbrace{u_{cute}^T v_{cat}}_{\text{positive}}) - \sum_{w \in \{w_{i_1}, \dots, w_{i_K}\}} \log \sigma(\underbrace{-u_w^T v_{cat}}_{\text{negative}})$$

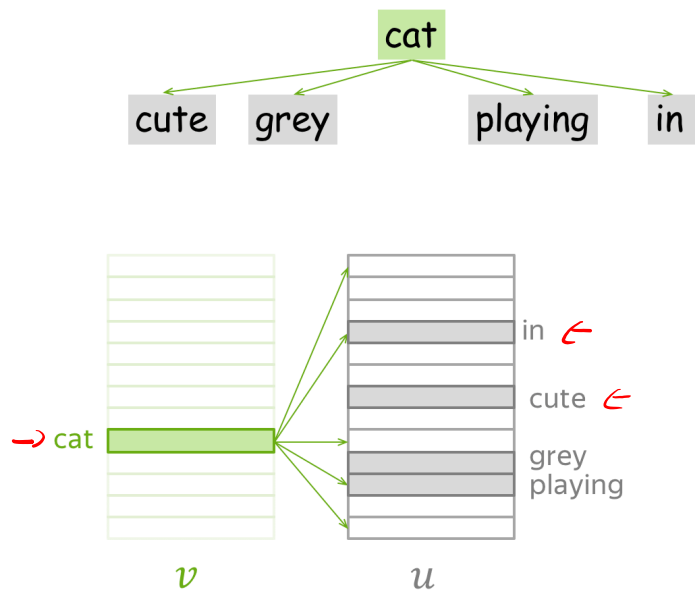


Table of Content

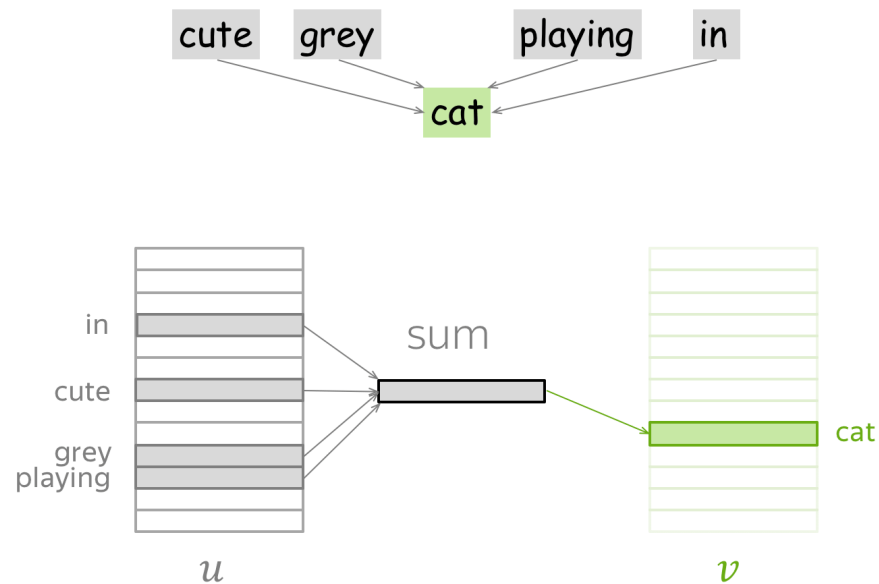
- Organizational matters
- Preprocessing pipeline
- But what is a Word Embedding?
- Count-based (pre-neural) approaches
- Word2vec
 - Idea behind
 - Objective function
 - Training procedure
 - Negative Sampling
 - **Skip-Gram vs. CBOW**
 - GloVe
- Useful facts

There are two ways to train the model

... I saw a cute grey cat playing in the garden ...



Skip-Gram: from **central** predict context
(one at a time)



CBOW: from sum of context predict **central**



Table of Content

- Organizational matters
- Preprocessing pipeline
- But what is a Word Embedding?
- Count-based (pre-neural) approaches
- Word2vec
 - Idea behind
 - Objective function
 - Training procedure
 - Negative Sampling
 - Skip-Gram vs. CBOW
 - **GloVe**
- Useful facts



Word2vec

GloVe

We can merge the two world views

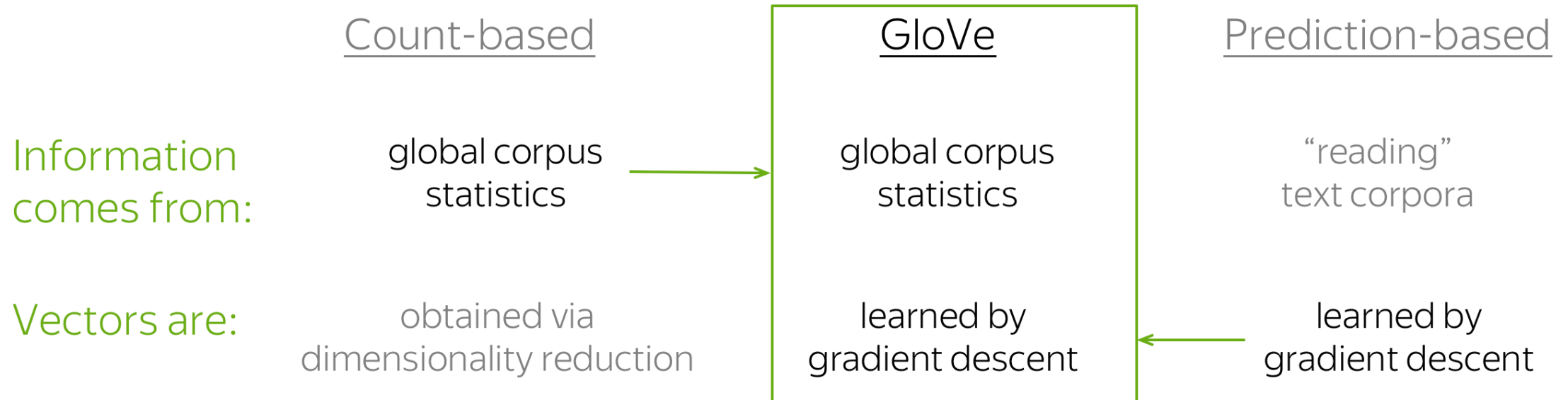




Table of Content

- Organizational matters
- Preprocessing pipeline
- But what is a Word Embedding?
- Count-based (pre-neural) approaches
- Word2vec
- **Useful facts**



- Normalize vectors due to cosine similarities nuances before moving embeddings to memory

$$\left\{ \begin{array}{l} \langle u, v \rangle = \underline{|u| |v|} \cos \varphi = \cos \varphi \\ |u| = 1 \\ |v| = 1 \end{array} \right.$$

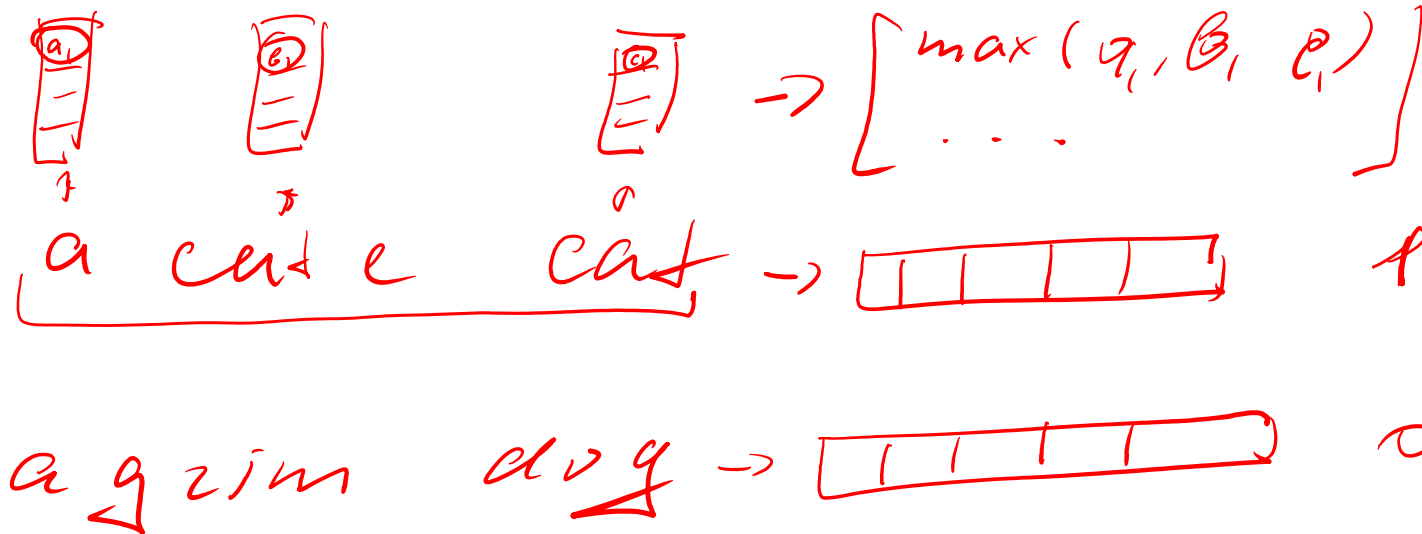


- Normalize vectors due to cosine similarities nuances before moving embeddings to memory
- The context for antonyms is very similar, hence embeddings for them are close



- Normalize vectors due to cosine similarities nuances before moving embeddings to memory
- The context for antonyms is very similar, hence embeddings for them are close
- Window size of a context determines the kind of relations one captures

- Normalize vectors due to cosine similarities nuances before moving embeddings to memory
- The context for antonyms is very similar, hence embeddings for them are close
- Window size of a context determines the kind of relations one captures
- We can aggregate individual word-level embeddings to represent a semantic of a collection of words



- Embeddings learned with word2vec lie in a linear well-explainable space
- Similar languages preserve the form of the space accurate to linear transformations

semantic: $v(\text{king}) - v(\text{man}) + v(\text{woman}) \approx v(\text{queen})$

syntactic: $v(\text{kings}) - v(\text{king}) + v(\text{queen}) \approx v(\text{queens})$

